prolog

1. Write a Prolog program to merge two ordered list to generate a third ordered list.

```
merge([],L,L).
merge(L,[],L).

merge([H1|T1],[H2|T2],[H1|T3]):-
H1=<H2,
merge(T1,[H2|T2],T3).

merge([H1|T1],[H2|T2],[H2|T3]):-
H1>H2,
merge([H1|T1],T2,T3).

?- merge([1,3,5], [2,4,6], X).
```

3. Write a Prolog program to add an element in a last position in a given list.

```
add_last([], X, [X]).
add_last([H|T], X, [H|R]) :-
add_last(T, X, R).

?- add_last([1,2,3], 4, L).
```

4. Write a Prolog program to check whether a year is a leap year or not.

```
leap_year(Y):-
0 is Y mod 400,!.
leap_year(Y):-
0 is Y mod 4,
0 =\= Y mod 100.

?- leap_year(2000).
```

6. Write a Prolog program to remove duplicates from a list.

```
remove_duplicates([],[]).

remove_duplicates([H|T],R):-
member(H,T),
remove_duplicates(T,R).

remove_duplicates([H|T],[H|R]):-
\+member(H,T),
```

```
remove_duplicates(T,R).
```

```
?- remove_duplicates([1,2,2,3,1,4], L).
```

7. Write a Prolog program to split a list into two halves.

```
split_list(L, Left, Right):-
length(L,N),
Half is N//2,
length(Left, Half),
append(Left, Right, L).
```

```
?- split_list([1,2,3,4,5,6], L1, L2).
```

8. Write a Prolog program to rotate a list N places to the left.

9. Write a Prolog program to find the union of two lists.

```
union([], L, L).
```

```
union([H|T],L2,R):-
member(H,L2),
union(T,L2,R).
union([H|T],L2,[H|R]):-
\+member(H,L2),
union(T,L2,R).
```

```
?- union([1,2,3], [3,4,5], R).
```

10. Write a Prolog program to replace all occurrences of an element in a list with another element.

```
replace([], _, _, []).
```

```
replace([X|T], X, Y, [Y|R]) :-
replace(T, X, Y, R).
```

```
replace([H|T], X, Y, [H|R]) :-
H \= X,
replace(T, X, Y, R).
```

```
?- replace([1,2,3,2,4], 2, 9, R).
```

Experiment 01 – Merging

1. **Q:** দুইটি লিস্ট কেন আগে সাজাতে হবে? না হলে কী হবে?

   Why must both input lists be sorted before merging? What happens if they are not?

   A: সাজানো না থাকলে সঠিক ক্রমে মিশানো যাবে না।

   If not sorted, merge will not produce correct order.

2. **Q:** merge/3-তে backtracking কীভাবে কাজ করে?

   Explain how backtracking behaves in your merge/3 predicate.

   A: প্রলগ সব সম্ভাব্য মিল খোঁজে, ব্যর্থ হলে ফিরে আসে।

   Prolog explores all possibilities and backtracks if no match is found.

3. **Q:** সময় জটিলতা কেন O(n+m)?

   What is the time complexity and why is it (O(n+m))?

   A: দুইটি লিস্ট একবার করে পড়া হয়, তাই n+m ধাপ লাগে।

   Each list is traversed once, so time complexity is O(n+m).

4. **Q:** যদি দুইটি উপাদান সমান হয়, কোন ক্লজটি প্রলগ ব্যবহার করে?

   How does Prolog choose between the two recursive clauses when elements are equal?

   A: সাধারণত প্রথম ক্লজটি আগে।

   Prolog chooses the first matching clause.

5. **Q:** প্রোগ্রাম ডুপ্লিকেট হ্যান্ডেল করতে পারে? উদাহরণ?

   Can your program handle duplicate elements correctly? Explain with an example.

   A: হ্যাঁ, উদা: [1,2] + [2,3] → [1,2,2,3]।

   Yes, duplicates are kept; e.g., [1,2] + [2,3] → [1,2,2,3].

6. **Q:** merge ডুপ্লিকেট ছাড়া করতে চাইলে কী করা যায়?

   How would you modify merge to remove duplicates while merging?

   A: member/2 দিয়ে চেক করে যোগ করতে হবে।

   Use member/2 to add only new elements.

7. **Q:** merge vs append + sort, কোনটা efficient?

Compare merging vs sorting using append + sort. Which is more efficient and why?

A: merge বেশি efficient কারণ একবারে মিশানো হয়।

Merge is more efficient; append+sort sorts after appending, slower.

Experiment 03 – Nth Element

1. **Q:** indexing কেন 1 থেকে শুরু?

   Why does indexing start from 1 in your implementation?

   A: প্রলগ সাধারণত 1-based indexing ব্যবহার করে।

   Prolog usually uses 1-based indexing.

2. **Q:** যদি N লিস্টের দৈর্ঘ্যের বেশি হয় কী হবে?

   What happens if N is larger than the list length?

   A: ফলাফল fail হবে।

   Returns failure.

3. **Q:** যদি N = 0 বা negative হয় কী হবে?

   What happens if N = 0 or negative?

   A: ভুল ইনপুট, fail হয়।

   Invalid input, fails.

4. **Q:** recursive stack কীভাবে কাজ করে?

   Explain the recursive stack behavior step-by-step.

   A: হেড skip করে N-1 করা হয়, শেষে base case এ যায়।

   Each call skips head and decrements N until base case.

5. **Q:** accessing Nth element সময় জটিলতা?

   What is the time complexity of accessing the Nth element?

   A: O(N) কারণ sequential traversal।

   O(N) due to sequential traversal.

6. **Q:** procedural language indexing থেকে কীভাবে আলাদা?

   How is this different from array indexing in procedural languages?

A: প্রলগ sequential, array direct access করে।

Prolog uses sequential traversal; arrays use direct indexing.

7. **Q:** arithmetic ছাড়া implement করা যাবে?

   Can you implement this without arithmetic (N1 is N-1)? How?

   A: হ্যাঁ, pattern matching বা counter variable দিয়ে।

   Yes, using pattern matching or a counter argument.

Experiment 04 – Leap Year

1. **Q:** কেন 400 rule আগে check করা হয়?

   Why must the 400 rule be checked before the 100 rule?

   A: 400 divisible year leap, 100 non-leap, সঠিক result এর জন্য।

   400 divisible years are leap, 100 divisible non-leap; order matters.

2. **Q:** clause order change করলে কী হয়?

   What happens if clause order is changed?

   A: ভুল output আসতে পারে।

   Wrong output may occur.

3. **Q:** mod operator এর কাজ কী?

   Explain the role of the mod operator in Prolog.

   A: divisibility চেক করতে।

   To check divisibility.

4. **Q:** logical failure কীভাবে treat করে Prolog?

   How does Prolog treat logical failure in leap year rules?

   A: fail করলে backtrack হয়।

   Failure triggers backtracking.

5. **Q:** এক বছরের জন্য multiple rules succeed করতে পারে?

   Can multiple rules succeed for one year? Why or why not?

   A: না, first matching rule ধরা হয়।

   No, only first matching rule succeeds.

6. **Q:** if–then–else দিয়ে implement করা যায় কীভাবে?

   How would you implement this using if–then–else (→ ;)?

   A: (Condition → leap ; non-leap)।

   Use (Condition → leap ; non-leap).

7. **Q:** Prolog procedural language থেকে কেন বেশি suitable?

   Why is Prolog more suitable for rule-based logic than procedural languages here?

   A: সহজে rule লিখতে ও check করতে পারে।

   Easier to express and check rules.

Experiment 06 – Duplicate Removal

1. **Q:** কেন member/2 ব্যবহার করা হয়?

   Why is member/2 used in this implementation?

   A: চেক করতে যে element আগেই আছে কি না।

   To check if an element already exists.

2. **Q:** duplicate removal এর time complexity কী?

   What is the time complexity of this duplicate removal method?

   A: $O(n^2)$ কারণ প্রতিটি element check হয়।

   $O(n^2)$ because each element is checked.

3. **Q:** original order কি preserve হয়? কেন?

   Does the program preserve the original order? Why?

   A: হ্যাঁ, কারণ আমরা sequentially append করি।

   Yes, because elements are added in sequence.

4. **Q:** + member(...) না দিলে কী হবে?

   What happens if you remove the negation + member(...)?

   A: ডুপ্লিকেট থাকবে।

   Duplicates will remain.

5. **Q:** backtracking duplicate detection এ কীভাবে প্রভাব ফেলে?

   How does backtracking affect duplicate detection?

A: সব সম্ভাবনা পরীক্ষা হয়, duplicate skip হয়।

All possibilities are checked; duplicates are skipped.

6. **Q:** sorting দিয়ে optimize করা যায় কীভাবে?

   How would you optimize this using sorting?

   A: লিস্ট sort করে neighbor elements compare করলে O(n log n)।

   Sort list and compare neighbors to reduce complexity.

7. **Q:** list_to_set/2 এর সাথে compare করো।

   Compare this approach with set operations in Prolog (list_to_set/2).

   A: list_to_set সহজ এবং efficient।

   list_to_set is simpler and more efficient.

Experiment 07 – Split List

1. **Q:** কেন length আগে calculate করা হয়?

   Why do we calculate the length first before splitting?

   A: split point নির্ধারণ করার জন্য।

   To determine the split point.

2. **Q:** odd length list হলে কী হয়?

   What happens if the list has odd length?

   A: এক list বড় হবে।

   One list will have one extra element.

3. **Q:** split_at/4 কিভাবে recursive কাজ করে?

   Explain how split_at/4 works recursively.

   A: head remove করে count reduce করে দুই ভাগে ভাগ করে।

   Removes head, decrements count, splits recursively.

4. **Q:** splitting এর time complexity কত?

   What is the time complexity of splitting?

   A: O(n) কারণ সব element একবার process হয়।

   O(n) because each element is processed once.

5. **Q:** length compute না করে split করা যায় কি?

   Can we split without computing length first?

   A: হ্যাঁ, counters বা recursion দিয়ে।

   Yes, using recursion or counters.

6. **Q:** কেন দুই output list লাগে?

   Why do we need two output lists instead of modifying one?

   A: Prolog immutable lists, নতুন lists তৈরি করতে হয়।

   Lists are immutable; need to create new lists.

7. **Q:** divide-and-conquer এর সাথে কী সম্পর্ক?

   How does this relate to divide-and-conquer algorithms like Merge Sort?

   A: list split করে recursive solve করা হয়।

   Splitting lists is the first step in divide-and-conquer.

Experiment 09 – Union of Lists

1. **Q:** কেন member/2 দিয়ে duplicate check করতে হয়?

   Why must duplicates be checked using member/2?

   A: এক element একবারই রাখতে।

   To ensure each element appears only once.

2. **Q:** union কি order preserve করে?

   Does union preserve order of elements? Explain.

   A: হ্যাঁ, প্রথম list elements প্রথমে থাকে।

   Yes, elements from the first list appear first.

3. **Q:** যদি উভয় list এর ভিতরেই duplicate থাকে কী হয়?

   What happens if both lists contain duplicates internally?

   A: duplicates skip হয়, unique element থাকে।

   Duplicates are skipped; only unique elements remain.

4. **Q:** union vs append + remove_duplicates compare করো।

   Compare union with append + remove_duplicates.

A: union সহজ ও readable।

Union is simpler and more readable.

5. **Q:** union এর time complexity কত?

   What is the time complexity of union?

   A: O(n*m) sequential member check।*
   ***O(n*m) due to sequential member checks.**

6. **Q:** Prolog sets vs lists কিভাবে treat করে?

   How does Prolog treat sets differently from lists?

   A: sets unique elements রাখে, lists allow duplicates।

   Sets contain unique elements; lists allow duplicates.

7. **Q:** intersection বা difference implement কিভাবে?

   How would you implement intersection or difference using similar logic?

   A: member/2 দিয়ে check করে add/remove করতে হবে।

   Use member/2 to check presence and add/remove elements.

Experiment 10 – Replace Elements

1. **Q:** কেন দুই recursive clause লাগে (match ও non-match)?

   Why do we need two recursive clauses (match and non-match)?

   A: match হলে replace, না হলে original keep করা।

   To replace matching elements and keep non-matching ones.

2. **Q:** target element না থাকলে কী হয়?

   What happens if the target element does not exist?

   A: কোনো change হয় না, original list থাকে।

   No change occurs; original list remains.

3. **Q:** replacement এর time complexity কত?

   What is the time complexity of replacement?

   A: O(n) কারণ সব element check হয়।

   O(n) because each element is checked.

4. **Q:** predicate কি bidirectional কাজ করে?

   Can this predicate work bidirectionally?

   A: হ্যাঁ, input/output swap করা যায়।

   Yes, can work in both directions.

5. **Q:** কোন clause আগে ব্যবহার হবে কিভাবে decide হয়?

   How does Prolog decide which clause to use first?

   A: top-down, first matching clause নির্বাচন হয়।

   Top-down; first matching clause is used.

6. **Q:** একসাথে multiple elements replace করতে চাইলে?

   How would you replace multiple different elements simultaneously?

   A: maplist বা recursive replace list দিয়ে।

   Use maplist or recursive replacement with lists.

7. **Q:** maplist/3 এর সাথে compare করো।

   Compare this approach with using maplist/3.

   A: maplist আরও concise ও readable।

   maplist is more concise and readable.